Inference-Time Latent Memory Management for Long-Horizon LLM Agents

Anonymous submission

Abstract

Generated latent memories can encode useful intermediate reasoning signals in LLM inference, but they are typically transient: once produced, there is no persistent mechanism for deciding which representations should remain available and be reused in later reasoning. We introduce an inference-time mechanism that rretrieves previously generated latent memories to regenerate query-conditioned latent representations for reasoning. A session-local memory bank provides bounded storage and retrieval for this mechanism without updating model parameters. The bank manages generated latent memories through bounded storage, relevance-based retrieval, thread updates, and capacity-aware replacement. On the EventQA subset of MemoryAgentBench, the latent memory manager improves exact match and reduces parser-format failures while maintaining runtime close to that of the matched no-bank path. These findings suggest that historical latent memories are effective when they condition query-specific latent regeneration, providing a potential alternative to repeatedly exposing long textual histories.

Introduction

Large language model (LLM)-based agents are evolving from single-turn text generators into interactive systems that can reason, use tools, maintain memory, and make decisions over extended interactions. A common abstraction decomposes their capabilities into reasoning or planning, memory, and tool use. Reasoning supports problem decomposition and decision making, memory preserves prior observations, task states, and experiences, and tool use connects the agent to external environments, APIs, databases, and other functional modules (Zhao, Jin, and Cheng 2023; Wang et al. 2024a; Zhang et al. 2025).

These components, however, are not independent. Both reasoning and tool use are fundamentally grounded in memory. Humans rarely solve a problem entirely from scratch; instead, they recall similar experiences, methods, tools, and previous failures while reasoning about the current situation. If a person used a tool yesterday but cannot remember either its existence or how to operate it today, the tool can no longer help solve the problem. The same limitation applies to LLM agents. An agent may have access to many tools, but without remembering when a tool should be invoked, which tool is appropriate, and how it should be used, tool availability alone does not guarantee successful task completion. From this perspective, tool use can be viewed as a form of procedural memory: the agent retrieves a relevant procedure and applies it to the current problem. Memory is therefore not merely a passive store of facts, but a foundational mechanism supporting reasoning, tool selection, task continuity, and long-horizon adaptation.

Memory representations in LLM agents can be broadly divided into explicit and latent forms. Explicit memory preserves information as natural-language text, structured records, or retrieved documents. Such representations are interpretable, editable, and easy to inspect, but they consume context capacity whenever they are presented to the model and are often only loosely coupled with its internal reasoning process. This limitation becomes particularly important in long-context settings. A complete interaction history may be too large to include at every inference step. Moreover, even when a model processes the history once, the useful intermediate representations produced during that computation are typically transient. Later queries must therefore recompute them from the original context or rely on another representation of the past.

Latent-memory methods offer one route to persistent internal state. Transformer-XL and Compressive Transformer reuse or compress hidden states across segments (Dai et al. 2019; Rae et al. 2020). Memorizing Transformers and CAMELoT retrieve stored internal activations or associative memories (Wu et al. 2022; He et al. 2024). MEMORYLLM and Titans introduce learned latent-memory mechanisms that maintain or update internal memory during inference or test-time adaptation (Wang et al. 2024b; Behrouz, Zhong, and Mirrokni 2025). Together, these approaches establish that internal representations can support information reuse beyond a single local context.

This work examines a complementary operational question: how should an agent manage latent memory tokens that are dynamically generated during reasoning? MemGen synthesizes latent memory tokens through a Trigger–Weaver–Reasoner pathway (Zhang, Fu, and Yan 2025). However, transient generation alone does not provide an ex-

plicitly managed store through which these tokens can persist and be selectively reused. As a result, latent representations derived from earlier parts of a long context may become unavailable to later reasoning steps, limiting their ability to support sustained information reuse over long horizons. We therefore study generated latent-token memory management: how generated memories should be stored, retrieved, updated, replaced, reset, and assigned to a bounded inference session.

We introduce an inference-time, session-local latent memory bank that extends MemGen by managing the latent memories generated during long-context reasoning. The bank stores generated latent memories during context processing and retrieves relevant memories to condition query-time latent-memory generation. We evaluate the proposed mechanism on EventQA, a long-context event reasoning task from MemoryAgentBench (Hu, Wang, and McAuley 2026).

The paper makes three contributions:

1. We introduce an inference-time, session-local latent memory management mechanism that extends MemGen without modifying model parameters.
2. We design a bounded latent memory bank with retrieval-conditioned reuse, similarity-based thread updates, capacity-aware replacement, and session-level reset operations.
3. We evaluate the proposed mechanism on EventQA through repeated runs, explicit textual-memory comparisons, query-time ablations, and efficiency analysis.

Related Work

Memory Representations and Management in LLM Agents

Memory is commonly treated as a core component of LLM agents alongside planning and tool use (Zhao, Jin, and Cheng 2023; Wang et al. 2024a). Existing systems differ in both their representational substrate and their management policy (Zhang et al. 2025). Explicit memory stores text, structured records, summaries, or retrieved documents. These forms are inspectable and can preserve source-level evidence, but they occupy context space when exposed to the model. Latent memory instead retains internal activations, learned states, or memory tokens. It is more tightly coupled to model computation, but its content is harder to inspect and verify.

This distinction does not by itself define a complete memory system. A long-horizon agent also needs rules for writing, selecting, updating, replacing, and clearing stored information. Our work focuses on these lifecycle operations for latent memories generated by an existing model. The contribution is therefore a management layer over generated latent state, rather than a new taxonomy of agent memory.

Recurrent and Retrieved Latent State

Several architectures preserve hidden state beyond a local segment. Transformer-XL reuses cached hidden states through segment-level recurrence (Dai et al. 2019), while Compressive Transformer retains older states in a compressed form (Rae et al. 2020). Recurrent Memory Transformer passes dedicated memory tokens between segments (Bulatov, Kuratov, and Burtsev 2022), and Infini-attention integrates compressive memory into the attention mechanism (Munkhdalai, Faruqui, and Gopal 2024). These methods embed persistence into the model's recurrent computation and typically require architecture-specific training.

Other approaches retrieve stored internal representations. Memorizing Transformers performs approximate nearest-neighbour lookup over a non-differentiable store of internal key-value pairs (Wu et al. 2022). CAMELoT attaches a training-free consolidated associative memory to a frozen language model (He et al. 2024). MEMORYLLM maintains a self-updatable latent memory pool (Wang et al. 2024b), and M+ combines latent memory with a trained retriever for longer retention (Wang et al. 2025). Titans instead learns a neural long-term memory that is updated at test time (Behrouz, Zhong, and Mirrokni 2025).

Our method shares the goal of persistent internal state but differs in its integration point. It does not add a new recurrent architecture or train a new memory module.

Explicit Memory and Long-Horizon Evaluation

Explicit-memory systems provide an important comparison because they retain readable evidence. Retrieval-augmented generation supplies selected documents to a parametric model (Lewis et al. 2020), while MemLLM trains a model to interact with an explicit read-write memory (Modarressi et al. 2025). MemGPT manages multiple memory tiers as virtual context for long-running interactions (Packer et al. 2023).

Long-horizon memory benchmarks also impose different evidence contracts. LongMemEval tests sustained conversational memory, including temporal reasoning, knowledge updates, and abstention (Wu et al. 2025). LoCoMo evaluates question answering, summarization, and dialogue generation over long conversations (Maharana et al. 2024). We focus on EventQA in MemoryAgentBench (Hu, Wang, and McAuley 2026), which tests event-centric reasoning over incrementally presented long contexts and directly supports our study of session-local memory construction and later retrieval interactions.

Generated Latent Memory in MemGen

MemGen dynamically generates machine-native memory during reasoning (Zhang, Fu, and Yan 2025). A memory Trigger decides when augmentation is needed, and a Weaver constructs a latent token sequence that supports the Reasoner. This design makes memory generation conditional on the current reasoning state.

We build on this mechanism rather than replace it. Our question begins after a Weaver output has been generated. We ask who owns that memory, how long it should remain available, and which memories a later query should retrieve. We also define how a bounded store updates or replaces its contents.

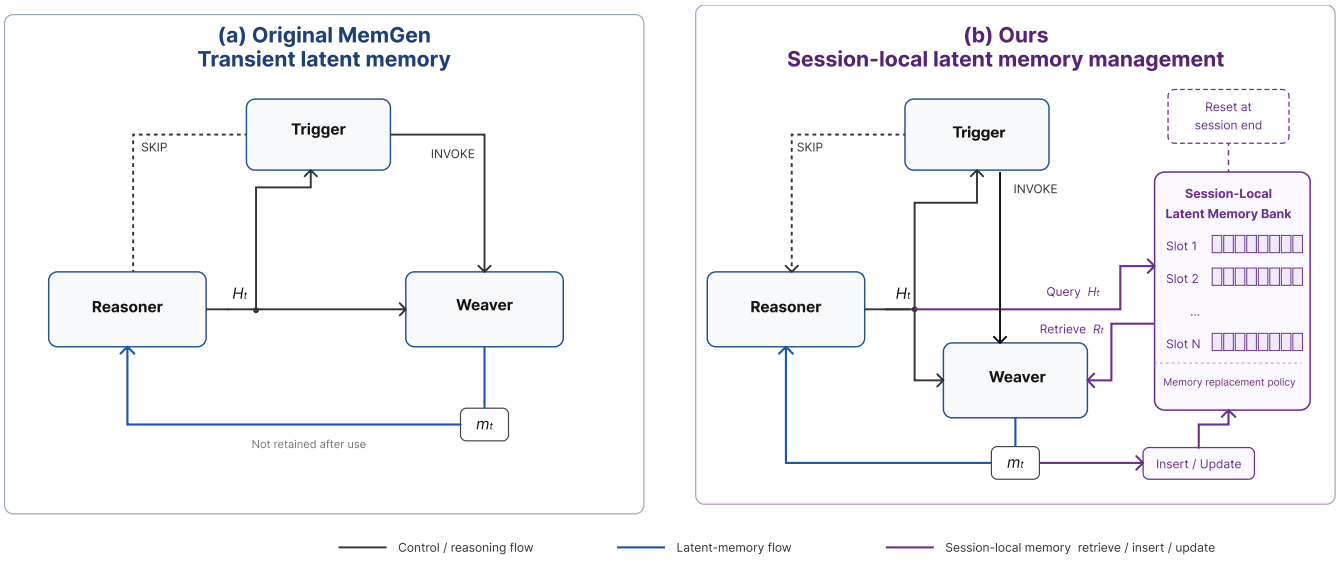

Figure 1: Comparison between transient latent-memory use in original MemGen and our session-local latent memory management mechanism. In the original pathway, the Weaver-generated latent memory m_t is consumed locally and is not retained. Our method retrieves prior session-local latent memories R_t to condition the Weaver and writes the newly generated m_t back to a bounded memory bank for later reuse.

Method

Overview

MemGen generates latent memories through a Trigger–Weaver–Reasoner pathway (Zhang, Fu, and Yan 2025). During inference, the Trigger determines whether latent-memory augmentation should be invoked. When invoked, the Weaver transforms the current Reasoner hidden state into a latent memory, which is subsequently injected into the Reasoner.

In the original pathway, each generated latent memory is transient and is not explicitly retained for later reasoning. We extend this pathway with a session-local latent memory bank. The bank stores Weaver-generated memories from earlier context-processing steps, retrieves memories relevant to the current reasoning state, and returns them to the Weaver. The Weaver therefore conditions on both the current hidden state and retrieved historical latent support before generating a new latent memory for the Reasoner.

The method changes only the inference flow. The Trigger, Weaver, and Reasoner remain frozen, and no additional training objective or cross-session memory is introduced. The bank belongs to a single inference session and is reset before an independent context is processed.

Session-Local Latent Memory Bank

Let H_t denote the Reasoner's hidden state at inference step t. When the Trigger invokes latent-memory augmentation, the query representation is obtained by mean-pooling the most recent L hidden states:

$$q_t = \mathrm{MeanPool}\left(H_t[-L:]\right).$$

The memory bank stores previously generated latent memories as

$$\mathcal{M}_t = \left\{\left(\widetilde{m}_i, k_i, r_i^{\mathrm{last}}\right)\right\}_{i=1}^{N_t},$$

where $\widetilde{m}_i$ is a detached Weaver-space latent memory stored in the bank, k_i is its retrieval key, and r_i^{last} records the global retrieval count at which $\widetilde{m}_i$ was most recently retrieved.

When a new latent memory is generated by the Weaver, it is detached from the computation graph and transferred to CPU storage:

$$\widetilde{m}_t = \mathrm{detach}(m_t).\,\mathrm{cpu}().$$

The key of the newly generated memory is then computed from the stored representation:

$$k_t = \mathrm{MeanPool}(\widetilde{m}_t).$$

The memory bank is session-local, does not share slots across contexts, and is reset before processing an independent context. Its size is bounded by a maximum capacity C.

Retrieval-Conditioned Latent Generation

For each stored memory, the bank computes a retrieval score that combines semantic similarity with retrieval-based temporal decay:

$$s_i(q_t) = \mathrm{cosine}(q_t, k_i)\exp\left(-\alpha\Delta r_i\right), \qquad \Delta r_i = r_t - r_i^{\mathrm{last}},$$

where r_t is the current global retrieval count and α controls the decay strength. This decay favors memories that have been retrieved more recently.

The retrieval candidate set is

$$\mathcal{C}_t = \{m_i \in \mathcal{M}_t \mid s_i(q_t) \geq \tau_r\},$$

where τ_r is the retrieval threshold. The bank returns at most the K highest-scoring candidates:

$$R_t = \mathrm{TopK}(\mathcal{C}_t, K).$$

If $\mathcal{C}_t = \varnothing$, the bank returns $R_t = \varnothing$. Whenever $m_i \in R_t$, its last-retrieval count is updated as

$$r_i^{\mathrm{last}} \leftarrow r_t.$$

The central difference from the original MemGen pathway is that the retrieved memories condition the Weaver rather than being directly injected into the Reasoner. When the Trigger emits `INVOKE`, the inference pathway is

$$m_t = \mathrm{Weaver}(H_t, R_t), \qquad y_t = \mathrm{Reasoner}(H_t, m_t).$$

When $R_t = \varnothing$, the Weaver follows the original generation path using only H_t. When the Trigger emits `SKIP`, the Reasoner proceeds without latent-memory augmentation:

$$y_t = \mathrm{Reasoner}(H_t).$$

This Weaver-mediated integration consolidates the current reasoning state and retrieved historical support into a newly generated latent memory before Reasoner inference. It therefore avoids directly exposing multiple retrieved memories to the Reasoner while preserving the original Weaver-to-Reasoner interface.

Memory Update and Capacity Management

Whenever the Weaver is invoked, the newly generated memory is detached and transferred to CPU storage. Its key is computed from the stored representation:

$$\widetilde{k}_t = \mathrm{MeanPool}(\widetilde{m}_t).$$

If the bank is empty, the detached memory is inserted directly:

$$\mathcal{M}_{t+1} = \{(\widetilde{m}_t, k_t, r_t)\}.$$

Otherwise, the bank identifies the stored memory most similar to the new candidate:

$$j = \arg\max_i \mathrm{cosine}(k_t, k_i), \qquad u_t = \mathrm{cosine}(k_t, k_j).$$

The update threshold τ_u determines whether the new candidate refreshes an existing latent-memory thread or creates a new one. If

$$u_t \geq \tau_u,$$

the most similar slot j is refreshed:

$$\mathcal{M}_{t+1} = \left(\mathcal{M}_t \setminus \left\{\left(m_j, k_j, r_j^{\mathrm{last}}\right)\right\}\right) \cup \{(\widetilde{m}_t, k_t, r_t)\}.$$

If $u_t < \tau_u$, the candidate is treated as a distinct latent-memory thread. When the bank has not reached capacity, the new slot is appended:

$$\mathcal{M}_{t+1} = \mathcal{M}_t \cup \{(\widetilde{m}_t, k_t, r_t)\}.$$

When the bank has reached capacity, the slot with the largest retrieval age is selected for eviction:

$$e = \arg\max_i \left(r_t - r_i^{\mathrm{last}}\right).$$

The selected slot is then replaced by the new memory:

$$\mathcal{M}_{t+1} = \left(\mathcal{M}_t \setminus \left\{\left(m_e, k_e, r_e^{\mathrm{last}}\right)\right\}\right) \cup \{(\widetilde{m}_t, k_t, r_t)\}.$$

Thus, τ_r controls whether a stored memory can be retrieved, whereas τ_u controls whether a generated memory refreshes an existing thread or creates a distinct one. The bounded-capacity policy preferentially preserves memories that have been retrieved more recently. The complete inference procedure, including retrieval, latent-memory generation, and memory update operations, is provided in Appendix.

Experiments

Research Questions

We organize the experiments around four research questions:

- **RQ1: Main effectiveness.** Does persistent, session-local reuse of generated latent memories improve long-context reasoning performance?
- **RQ2: Comparison with explicit memory.** Can explicit textual-memory controls, including recursive summarization and sparse retrieval, reproduce the effectiveness of the latent memory manager?
- **RQ3: Query-time memory integration.** Which components of query-time latent-memory retrieval and integration are necessary for the observed performance gains?
- **RQ4: Efficiency.** What inference-time latency and GPU-memory costs does the latent memory manager introduce relative to the evaluated no-bank and explicit-memory controls?

Experimental Setup

Benchmark and evaluation protocol. We evaluate on EventQA (Hu, Wang, and McAuley 2026), which tests event-centric question answering over long contexts. Each evaluation pass contains five contexts with 100 questions per context (500 questions in total), following the original context partitioning and evaluation protocol. All methods and ablations are evaluated over five complete passes with different base seeds, and we report mean and population standard deviation.

For the latent-memory condition, the memory bank is constructed during context processing and frozen before question answering. Query-time evaluation is read-only: questions may retrieve from the frozen bank but cannot create,

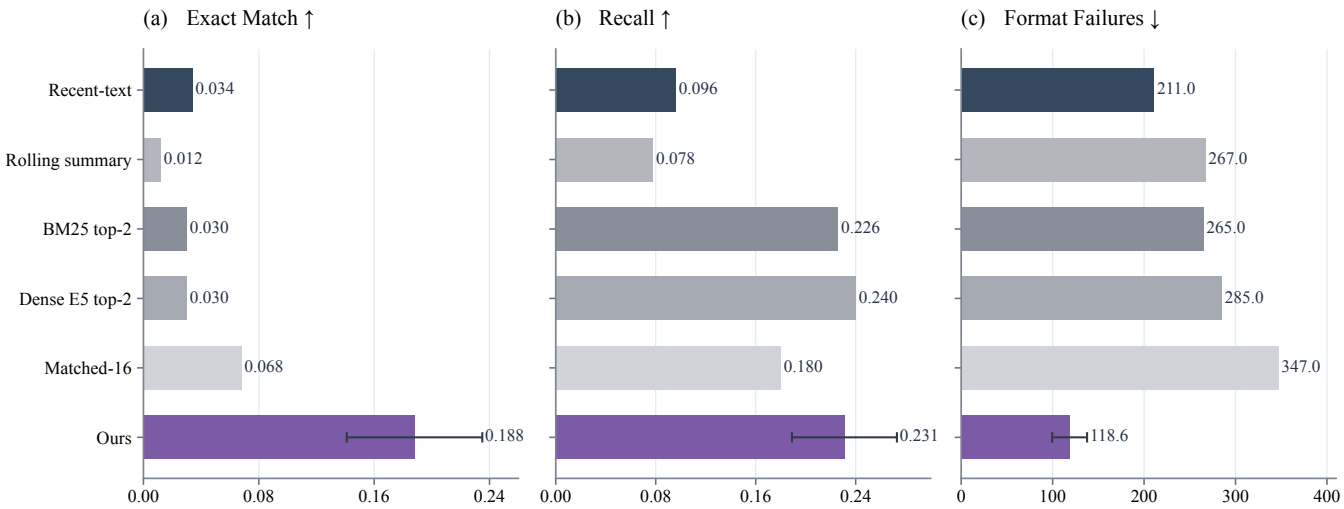

Figure 2: Overall EventQA effectiveness across the capacity-max MemGen recent-text baseline, explicit textual-memory controls, and the latent memory manager. Bars report means over five complete process-level passes, and error bars denote population standard deviations. Higher is better for exact match and recall, whereas lower is better for format failures.

update, replace, or evict memory slots. The same frozen snapshot is restored for all questions within a context, preventing cross-question memory accumulation.

Unless otherwise stated, we use a 16-slot bank with retrieval threshold 0.05, update threshold 0.10, top-$k = 2$, and temporal-decay coefficient $\alpha = 0.05$. The Trigger, Weaver, and Reasoner parameters remain frozen throughout evaluation. Additional implementation details are provided in Appendix.

Evaluation metrics. We use the default EventQA prompt and unchanged parser and scorer without output post-processing. Following the benchmark evaluation, substring exact match (EM) is the primary metric. We additionally report answer recall, which measures whether the gold event appears in the raw output, and format failures, which count unparsable responses.

Compared Methods

MemGen recent-text baseline (capacity-max). To construct a capacity-limited textual-history control for MemGen without a persistent latent bank, we use the same MemGen checkpoint with the bank disabled and provide raw textual history directly in the prompt. Complete EventQA histories exceed the model's 32,768-token context capacity. We therefore use a capacity-limited proxy for full-history conditioning rather than claiming an official full-history MemGen baseline. For each question, we prepend the longest chronologically ordered suffix of the event history that fits within the available input budget, followed by the unchanged EventQA query template. This baseline tests whether a context-capacity-maximized suffix of recent raw text can substitute for persistent latent-memory reuse.

Rolling text summary. This same-model explicit-memory control processes each EventQA context sequentially. For every context chunk, the generator receives the previous summary and the new chunk, greedily produces an updated summary, and persists at most 128 tokenizer tokens. After construction, the final summary is prepended to every question for that context. It remains fixed during question answering, and the latent bank is disabled throughout. It tests whether a compact natural-language summary can substitute for persistent latent memory.

BM25 retrieval. We construct a per-context lexical index over the EventQA chunks using Okapi BM25 (Robertson et al. 1994), with lower-cased alphanumeric terms, $k_1 = 1.5$, and $b = 0.75$. Each question retrieves the two highest-scoring chunks, with ties broken by original chunk order. Their full text is prepended to the unchanged EventQA query prompt, and the same generator answers with the latent bank disabled. This baseline tests whether sparse retrieval over explicit source text can reproduce the benefit of latent-memory retrieval.

Dense E5 retrieval. To test whether the sparse ranker limits its retrieval effectiveness, we construct an index over only the current EventQA context using a frozen local E5-base-v2 encoder. Each 4,096-token parent chunk is partitioned into non-overlapping windows of 500 tokens under the E5 tokenizer. The official EventQA question, including its candidate events, is encoded as the query. Each parent chunk is scored by the maximum cosine similarity between the query and any of its windows, and the two highest-scoring distinct parent chunks are selected. Their full text is prepended using the same retrieved-passage template as the BM25 baseline, and the same generator answers with the latent bank disabled. This baseline replaces the sparse BM25 ranker with dense E5 retrieval while preserving the source scope, top-k, text-injection procedure, generation settings, and evaluation protocol of the full-text BM25 control.

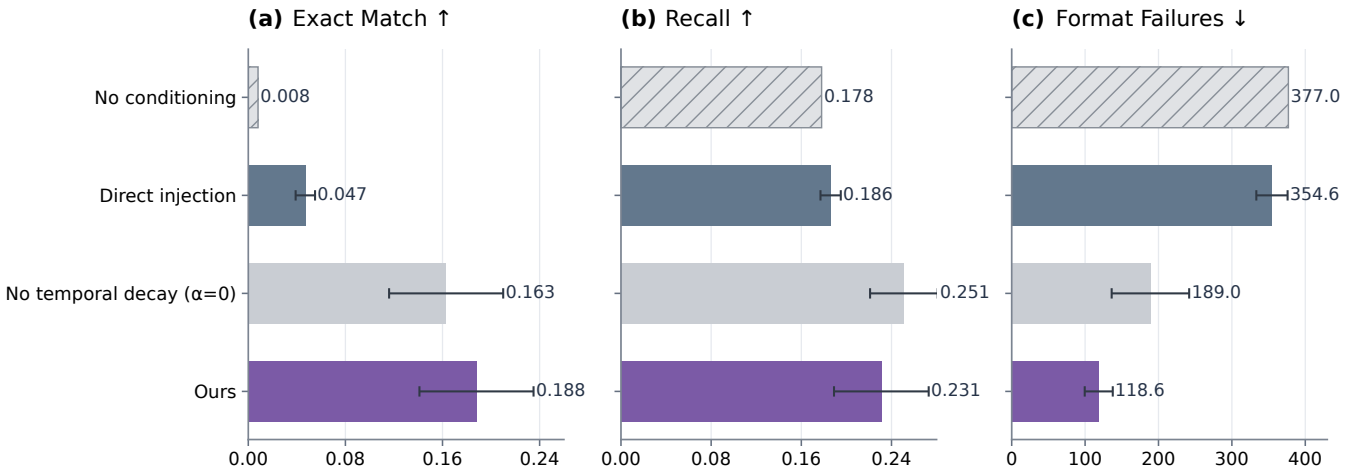

Figure 3: Ablation results. Bars report means over five complete process-level runs, and error bars denote population standard deviations. Higher is better for EM and recall, whereas lower is better for format failures.

Matched-16 BM25 retrieval. To control the amount of visible source text, this variant starts from the same top-two BM25 chunks but selects one query-overlap-scored eight-token window from each. The two windows are injected only when the rendered prompt adds exactly 16 source tokens; otherwise, the run is rejected. This control separates the effect of lexical retrieval from that of exposing a larger amount of source text.

All compared methods use the same base model, EventQA questions, default query prompt, benchmark parser and scorer, and 40-token answer-generation budget.

Results

Main Results

Figure 2 compares the latent memory manager with the capacity-max MemGen recent-text baseline and four explicit textual-memory controls. The latent memory manager achieves the strongest overall performance, reaching an EM of 0.188 and recall of 0.231, while reducing format failures to 118.6. In comparison, the capacity-max recent-text baseline attains only 0.034 EM and 0.096 recall, with 211.0 format failures.

The retrieval-based textual controls provide only partial benefits. BM25 top-2 reaches a recall of 0.226 but only 0.030 EM, while Dense E5 top-2 slightly improves recall to 0.240 without improving EM (0.030) and incurs more format failures (285.0). Thus, replacing the sparse BM25 ranker with dense retrieval improves the chance that the gold event appears in the output, but does not improve exact answer selection or output reliability. The matched-budget retrieval condition achieves the strongest textual-control EM at 0.068, whereas the rolling-summary control remains weaker at 0.012.

Overall, these results indicate that the improvement cannot be explained solely by providing more textual context, using a stronger retrieval ranker, retrieving relevant source passages, or compressing the history into an explicit summary. Instead, the results support the use of a session-local latent memory bank that retrieves historical latent support and integrates it through the Weaver before Reasoner inference.

Ablation Study

We evaluate three query-time ablations under the same EventQA protocol. *No retrieved-memory conditioning* performs retrieval but removes retrieved latents before Weaver generation. *Direct latent injection* bypasses Weaver-mediated integration by injecting the retrieved latent directly into the Reasoner. *No temporal decay* removes the retrieval inactivity factor by setting $\alpha = 0$ while preserving the remaining configuration.

As shown in Figure 3, removing retrieved-memory conditioning reduces EM to 0.008 and increases format failures to 377.0, despite executing the complete retrieval procedure. This shows that retrieval computation alone is insufficient without latent-memory integration through the Weaver.

Direct latent injection achieves only 0.047 ± 0.008 EM and 0.186 ± 0.009 recall, with 354.6 ± 21.3 format failures. Its gap from the full method indicates that direct reuse of retrieved latents cannot replace Weaver-mediated consolidation.

Removing temporal decay yields 0.163 ± 0.047 EM, 0.251 ± 0.030 recall, and 189.0 ± 52.8 format failures, showing that temporal weighting improves overall answer reliability despite slightly lower recall.

Additional hyperparameter sensitivity analyses, including bank capacity, retrieval depth, retrieval threshold, update threshold, and construction policies, are provided in the Appendix.

Efficiency Analysis

Figure 4 reports three independently launched, serialized complete-process runs on the same GPU. Each run covers five contexts and 500 questions with a single model load. Model-loading time is excluded, while method-specific preparation, including bank construction, summary generation, and retrieval-index construction, is included. We report

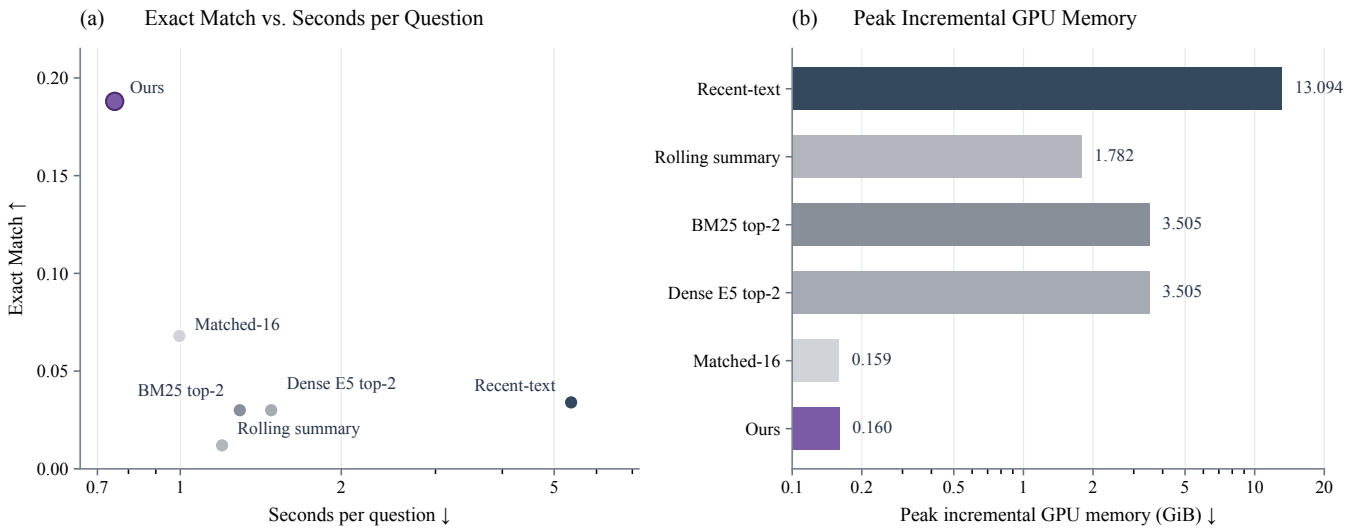

Figure 4: Effectiveness–efficiency comparison on EventQA. The left panel shows mean exact match against mean seconds per question, and the right panel shows the maximum observed peak incremental GPU-memory allocation. Timing means are computed over three complete-process runs; standard deviations are reported in the text. Horizontal axes use logarithmic scales.

mean $\pm$ population standard deviation over the three runs and the maximum observed peak incremental GPU-memory allocation.

The latent memory manager requires 377.06 ± 10.49 seconds end-to-end (0.754 ± 0.021 seconds per question) with 0.160 GiB peak incremental GPU memory. In comparison, the capacity-max MemGen recent-text baseline requires 2688.90 ± 9.50 seconds and 13.094 GiB, making the latent memory manager approximately $7.1\times$ faster and reducing incremental memory allocation by about $82\times$. The higher cost of the recent-text baseline results from repeatedly prefilling 32,256 source tokens for each question.

The rolling-summary, BM25 top-2, Dense E5 top-2, and Matched-16 controls are all slower than the latent memory manager. Dense E5 top-2 has slightly higher latency than BM25 top-2 while using the same peak allocation (3.505 GiB), indicating that dense encoding does not reduce the cost of full-text retrieval under this setup. The latent memory manager and Matched-16 have nearly identical peak allocations (0.160 versus 0.159 GiB), suggesting that their latency difference is primarily due to query-time computation rather than memory usage.

These measurements characterize inference costs under the evaluated EventQA protocol and do not imply universal efficiency advantages across models, hardware, encoders, or retrieval implementations.

Conclusion

We introduced an inference-time, session-local latent memory manager for MemGen that stores and reuses Weaver-generated memories without updating model parameters. On EventQA, the proposed mechanism improves exact match and reduces format failures, while outperforming the evaluated explicit textual-memory controls. Ablation studies show that the gains arise from conditioning Weaver generation on retrieved latent memories rather than from memory construction or retrieval computation alone. These results demonstrate the potential of session-local latent-memory reuse for long-context reasoning under the evaluated setting, while broader validation across models, tasks, and memory architectures remains future work.

Limitations. Our evaluation is limited to EventQA and a single MemGen checkpoint, so the results may not generalize to other models or memory tasks. Moreover, comparisons with other latent-memory architectures are beyond the scope of this work. The reported efficiency results are specific to the evaluated hardware and protocol.

References

Behrouz, A.; Zhong, P.; and Mirrokni, V. 2025. Titans: Learning to Memorize at Test Time. In *Advances in Neural Information Processing Systems*.

Bulatov, A.; Kuratov, Y.; and Burtsev, M. S. 2022. Recurrent Memory Transformer. In *Advances in Neural Information Processing Systems*, volume 35.

Dai, Z.; Yang, Z.; Yang, Y.; Carbonell, J.; Le, Q. V.; and Salakhutdinov, R. 2019. Transformer-XL: Attentive Language Models beyond a Fixed-Length Context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2978–2988. Association for Computational Linguistics.

He, Z.; Karlinsky, L.; Kim, D.; McAuley, J.; Krotov, D.; and Feris, R. 2024. CAMELoT: Towards Large Language Models with Training-Free Consolidated Associative Memory. In *ICML 2024 Workshop on Long-Context Foundation Models*.

Hu, Y.; Wang, Y.; and McAuley, J. 2026. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. In *The Fourteenth International Conference on Learning Representations*.

Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, 9459–9474.

Maharana, A.; Lee, D.-H.; Tulyakov, S.; Bansal, M.; Barbieri, F.; and Fang, Y. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 13851–13870. Association for Computational Linguistics.

Modarressi, A.; Köksal, A.; Imani, A.; Fayyaz, M.; and Schütze, H. 2025. MemLLM: Finetuning LLMs to Use Explicit Read-Write Memory. *Transactions on Machine Learning Research*.

Munkhdalai, T.; Faruqui, M.; and Gopal, S. 2024. Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention. *arXiv preprint arXiv:2404.07143*.

Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S. G.; Stoica, I.; and Gonzalez, J. E. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560*.

Rae, J. W.; Potapenko, A.; Jayakumar, S. M.; and Lillicrap, T. P. 2020. Compressive Transformers for Long-Range Sequence Modelling. In *International Conference on Learning Representations*.

Robertson, S. E.; Walker, S.; Jones, S.; Hancock-Beaulieu, M. M.; and Gatford, M. 1994. Okapi at TREC-3. In *Proceedings of the Third Text REtrieval Conference (TREC-3)*, 109–126. National Institute of Standards and Technology.

Wang, L.; Ma, C.; Feng, X.; Zhang, Z.; Yang, H.; Zhang, J.; Chen, Z.; Tang, J.; Chen, X.; Lin, Y.; Zhao, W. X.; Wei, Z.; and Wen, J.-R. 2024a. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science*, 18(6): 186345.

Wang, Y.; Gao, Y.; Chen, X.; Jiang, H.; Li, S.; Yang, J.; Yin, Q.; Li, Z.; Li, X.; Yin, B.; Shang, J.; and McAuley, J. 2024b. MEMORYLLM: Towards Self-Updatable Large Language Models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 50453–50466. PMLR.

Wang, Y.; Krotov, D.; Hu, Y.; Gao, Y.; Zhou, W.; McAuley, J.; Gutfreund, D.; Feris, R.; and He, Z. 2025. M+: Extending MemoryLLM with Scalable Long-Term Memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 63308–63323. PMLR.

Wu, D.; Wang, H.; Yu, W.; Zhang, Y.; Chang, K.-W.; and Yu, D. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *The Thirteenth International Conference on Learning Representations*.

Wu, Y.; Rabe, M. N.; Hutchins, D.; and Szegedy, C. 2022. Memorizing Transformers. In *International Conference on Learning Representations*.

Zhang, G.; Fu, M.; and Yan, S. 2025. MemGen: Weaving Generative Latent Memory for Self-Evolving Agents. *arXiv preprint arXiv:2509.24704*.

Zhang, Z.; Dai, Q.; Bo, X.; Ma, C.; Li, R.; Chen, X.; Zhu, J.; Dong, Z.; and Wen, J.-R. 2025. A Survey on the Memory Mechanism of Large Language Model-based Agents. *ACM Transactions on Information Systems*, 43(6): 1–47.

Zhao, P.; Jin, Z.; and Cheng, N. 2023. An In-depth Survey of Large Language Model-based Artificial Intelligence Agents. *arXiv preprint arXiv:2309.14365*.